

EXPERIMENT 04

BEST FIRST SEARCH

Artificial Intelligence / Python Practical

Experiment	04
Title	Implementation of Best First Search in Python
Language	Python
Search Type	Heuristic / Informed Search
Primary Data Structure	Priority Queue (Min-Heap)

1. Aim

To write and execute a Python program that implements the Best First Search algorithm using heuristic values to select the most promising node during graph traversal.

2. Objective

The objective is to understand how an informed search algorithm uses a heuristic function $h(n)$ to decide which node should be explored next. Best First Search gives priority to the node that appears closest to the goal according to its heuristic value.

3. Theory

Best First Search is an informed search technique. It selects the next node based on an evaluation value, commonly the heuristic $h(n)$. In the greedy version of Best First Search, the node with the smallest heuristic value is selected first.

A priority queue is used to keep the available nodes ordered according to their heuristic values. A visited set is maintained to avoid repeatedly processing the same node.

Heuristic function $h(n)$: $h(n)$ estimates how close node n is to the goal. For greedy Best First Search, a lower $h(n)$ is considered more promising.

4. Graph Used in the Practical

For this practical, the following example graph and heuristic values are used:

Node	Neighbours	$h(n)$
A	B, C	6
B	D, E	4
C	F, G	2

D	—	7
E	—	3
F	—	1
G	—	0

Start node: A Goal node: G

Graph relationships: $A \rightarrow B, C$; $B \rightarrow D, E$; $C \rightarrow F, G$.

5. Algorithm

1. Initialize a priority queue with the start node and its heuristic value.
2. Create an empty visited set.
3. Remove the node having the smallest heuristic value from the priority queue.
4. If the node has already been visited, skip it.
5. Mark the node as visited and add it to the traversal order.
6. If the current node is the goal, stop the search.
7. Otherwise, insert its unvisited neighbours into the priority queue using their heuristic values.
8. Repeat until the goal is found or the priority queue becomes empty.

6. Python Program

```
import heapq

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 2,
    'D': 7,
    'E': 3,
    'F': 1,
    'G': 0
}

def best_first_search(graph, heuristic, start, goal):
    priority_queue = [(heuristic[start], start)]
    visited = set()
    traversal = []

    while priority_queue:
        h_value, node = heapq.heappop(priority_queue)
```

```

    if node in visited:
        continue

    visited.add(node)
    traversal.append(node)

    if node == goal:
        break

    for neighbour in graph[node]:
        if neighbour not in visited:
            heapq.heappush(
                priority_queue,
                (heuristic[neighbour], neighbour)
            )

    return traversal

start = 'A'
goal = 'G'

result = best_first_search(graph, heuristic, start, goal)

print("Best First Search Traversal:")
print(" -> ".join(result))

```

7. Expected Output

```

Best First Search Traversal:
A -> C -> G

```

8. Step-by-Step Execution

Step	Current Node	Important Choice	Priority Reason
1	A	Add B, C	$h(C)=2$ is smaller than $h(B)=4$
2	C	Add F, G	$h(G)=0$ is smaller than $h(F)=1$
3	G	Goal found	$h(G)=0$

9. Time and Space Complexity

The exact complexity depends on the priority-queue implementation and graph representation. For a heap-based implementation, each queue insertion/removal involves logarithmic heap cost. A common graph-search bound is expressed as $O((V + E) \log V)$ when heap operations and a suitable visited structure are used, where V is the number of vertices and E is the number of edges. The space requirement is $O(V)$ for the visited set and priority queue in the graph-search setting.

10. Advantages

- Uses heuristic information to guide the search.
- Can reach a promising goal quickly when the heuristic is useful.
- Prioritizes nodes that appear closer to the goal.
- Can be implemented with a priority queue.

11. Disadvantages

- Its result depends strongly on the quality of the heuristic.
- Greedy Best First Search is not guaranteed to return an optimal path.
- It may explore an unhelpful branch if the heuristic is misleading.
- The priority queue can require significant memory for large search spaces.

12. Applications

- Route and path exploration
- Game and puzzle search
- Robotics and navigation
- Problem-solving systems
- Search spaces where heuristic estimates are available

13. Best First Search vs BFS vs A*

Feature	BFS	Best First Search	A*
Guidance	Level/depth	Heuristic $h(n)$	$g(n) + h(n)$
Typical structure	FIFO queue	Priority queue	Priority queue
Uses heuristic?	No	Yes	Yes
Optimality	For unweighted shortest paths	Not guaranteed	Under suitable conditions

14. Viva Questions and Answers

1. What is Best First Search?

It is an informed search algorithm that selects the most promising node according to a heuristic value.

2. What is a heuristic function?

A heuristic function $h(n)$ estimates how promising or close a node is to the goal.

3. Which data structure is commonly used?

A priority queue is commonly used so the node with the best priority can be selected.

4. What does greedy Best First Search use?

It primarily uses $h(n)$, the estimated cost from the current node to the goal.

5. Is Best First Search always optimal?

No. Greedy Best First Search is not guaranteed to find the optimal path.

6. What is the main advantage of Best First Search?

It can guide the search toward promising nodes instead of exploring nodes only by depth.

7. What is the difference between BFS and Best First Search?

BFS explores level by level using a FIFO queue, while Best First Search uses heuristic values and a priority queue.

8. What is the difference between Best First Search and A*?

Greedy Best First Search primarily uses $h(n)$, while A* uses $g(n) + h(n)$.

15. Conclusion

Best First Search was implemented in Python using a heuristic function and a priority queue. The experiment demonstrates how heuristic information can guide graph search toward a selected goal node. The example shows the traversal $A \rightarrow C \rightarrow G$ for the given graph and heuristic values.

16. Practical Record Note

This document is a clean, portfolio-ready practical record template. If your college-issued practical sheet uses a different graph, heuristic table, program, or output, those source-specific details should replace the corresponding sections before submission.